Human Verification: `Containers.FourMonoidal`

The Four Monoidal Structures on **Cont**

Machine-generated from `lean/FourMonoidal.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.FourMonoidal` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

The file completes the account of all four monoidal structures on **Cont**: the coproduct $(+, 0)$, the categorical product $(\times, 1)$, the Dirichlet tensor $(\otimes, y)$ (in `Dirichlet.lean`), and the sequential operator $(\triangleleft, I)$ (in `Monoidal.lean`). It also proves that the extension functor $[\![-]\!]$ is *strong* monoidal for $+$ and $\times$, and provides the comparison isomorphisms.

Contents

1 Overview **2**

2 Unit containers **2**
2.1 Definition: `Container.zero` . . . 2
2.2 Definition: `Container.one` . . . 2

3 The coproduct $(+, 0)$ **3**
3.1 Definition: `Container.coprod`$_2$ . . . 3
3.2 Theorem: `Container.coprod`$_2$`_id` . . . 3
3.3 Theorem: `Container.coprod`$_2$`_comp` . . . 3
3.4 Definition: `Container.coprodAssociator` . . . 4
3.5 Definition: `Container.coprodLeftUnitor` . . . 4
3.6 Definition: `Container.coprodRightUnitor` . . . 5
3.7 Theorem: `Container.coprodAssociator_naturality` . . . 5
3.8 Theorem: `Container.coprodLeftUnitor_naturality` . . . 5
3.9 Theorem: `Container.coprodRightUnitor_naturality` . . . 6
3.10 Theorem: `Container.coprodPentagon` . . . 6
3.11 Theorem: `Container.coprodTriangle` . . . 6
3.12 Definition: `contCoprodMonoidal` . . . 6

4 The categorical product $(\times, 1)$ **7**
4.1 Definition: `Container.prod`$_2$ . . . 7
4.2 Theorem: `Container.prod`$_2$`_id` . . . 7
4.3 Theorem: `Container.prod`$_2$`_comp` . . . 7
4.4 Definition: `Container.prodAssociator` . . . 8
4.5 Definition: `Container.prodLeftUnitor` . . . 8
4.6 Definition: `Container.prodRightUnitor` . . . 8
4.7 Theorem: `Container.prodAssociator_naturality` . . . 9

4.8 Theorem: `Container.prodLeftUnitor_naturality` . . . 9
4.9 Theorem: `Container.prodRightUnitor_naturality` . . . 9
4.10 Theorem: `Container.prodPentagon` . . . 9
4.11 Theorem: `Container.prodTriangle` . . . 10
4.12 Definition: `contProdMonoidal` . . . 10

5 The extension functor as a strong monoidal functor 10
5.1 Definitions and theorems: coproduct comparison . . . 10
5.2 Definition/Theorem: `Container.zeroExt` . . . 11
5.3 Definitions and theorems: product comparison . . . 11
5.4 Definitions and theorems: unit comparison for 1 . . . 12
5.5 Definitions: shared unit comparison $[\![y]\!]X \cong X$ . . . 13

1 Overview

The four monoidal structures on **Cont** are summarised by the table:

Structure	Shapes of $C \otimes D$	Positions of $C \otimes D$	Unit
coproduct $+$	$S \sqcup T$	$[P, Q]$ (by injection)	$0 = \emptyset \triangleleft \cdots$
product $\times$	$S \times T$	$P\,s \sqcup Q\,t$	$1 = \mathbf{1} \triangleleft \emptyset$
Dirichlet $\otimes$	$S \times T$	$P\,s \times Q\,t$	$y = \mathbf{1} \triangleleft \mathbf{1}$
sequential $\triangleleft$	$\Sigma\, s, P\,s \to T$	$\Sigma\, p, Q(f(p))$	$I = y$

Key observation. The product $\times$ and Dirichlet $\otimes$ have the same shapes $S \times T$ but differ only in their positions (sum vs. product). Yet the *extensions* invert the reading: $[\![C \times D]\!]X \cong [\![C]\!]X \times [\![D]\!]X$ (pointwise product), while $[\![C \otimes D]\!]X$ is *not* any pointwise combination of $[\![C]\!]X$ and $[\![D]\!]X$. This is the exponential law: $y^{A+B} \cong y^A \times y^B$, but $y^{A \times B}$ admits no such splitting.

Only `ContMonoidal` (sequential $\triangleleft$) is the canonical `MonoidalCategory Container` *instance*; the other three are named `def`s to avoid instance clashes.

2 Unit containers

2.1 Definition: `Container.zero`

Definition 2.1 (The empty container 0)**.** The **empty container** $0 = \emptyset \triangleleft (\text{elim})$ has no shapes and (vacuously) no positions. It is the unit for the coproduct $+$ and the initial object of **Cont**. Its extension $[\![0]\!]X$ is the empty type for every X.

Lean.
```
def Container.zero : Container where
  Shape := Empty
  Pos   := fun e ⇒ e.elim
```

Haskell.
```
zeroContainer :: Container Void p
zeroContainer = Container { shape = absurd, pos = absurd }
-- Extension: Ext ZeroContainer x  is  uninhabited.
```

2.2 Definition: `Container.one`

Definition 2.2 (The one-shape container 1)**.** The **one-shape, no-position container** $1 = \mathbf{1} \triangleleft \emptyset$ is the polynomial y^0. It is the unit for the categorical product $\times$ and the terminal object of **Cont**. Its extension $[\![1]\!]X$ is a singleton type for every X.

Lean.
```
def Container.one : Container where
  Shape := Unit
  Pos   := fun _ ⇒ Empty
```

Haskell.
```
oneContainer :: Container () Void
oneContainer = Container { shape = const (), pos = const absurd }
-- Extension: Ext OneContainer x  =~=  Unit.
```

3 The coproduct $(+, 0)$

3.1 Definition: `Container.coprod₂`

Definition 3.1 (Bifunctorial action of $+$)**.** The morphism action $\texttt{coprod}_2\,\mu\,\nu : C+D \Rightarrow C'+D'$ is derived from the coproduct's universal property: apply μ to C-shapes and ν to D-shapes via the injections, then co-pair.

Lean.
```
def Container.coprod2 {C C' D D' : Container}
    (mu : ContainerMorphism C C') (nu : ContainerMorphism D D') :
    ContainerMorphism (C.coprod D) (C'.coprod D') :=
  (mu.comp (Container.inl C' D')).desc (nu.comp (Container.inr C' D'))
```

Haskell.
```
coprod2 :: Morph c p c' p' → Morph d q d' q'
        → Morph (c `Coprod` d) (Either p q) (c' `Coprod` d') (Either
           p' q')
coprod2 mu nu = desc (mu `compM` inlMorph) (nu `compM` inrMorph)
```

3.2 Theorem: `Container.coprod₂_id`

Theorem 3.2 (Coproduct bifunctoriality: identity)**.** $\texttt{coprod}_2\,(\mathrm{id}_C)\,(\mathrm{id}_D) = \mathrm{id}_{C+D}$.

Lean.
```
theorem Container.coprod2_id (C D : Container) :
    Container.coprod2 (ContainerMorphism.id C) (ContainerMorphism.id D
       )
      = ContainerMorphism.id (C.coprod D)
```

Haskell.
```
-- coprod2 idMorph idMorph = idMorph
```

3.3 Theorem: `Container.coprod₂_comp`

Theorem 3.3 (Coproduct bifunctoriality: interchange)**.** $\texttt{coprod}_2\,(\mu\cdot\mu')\,(\nu\cdot\nu') = \texttt{coprod}_2\,\mu\,\nu \cdot \texttt{coprod}_2\,\mu'\,\nu'$.

Lean.
```
theorem Container.coprod2_comp {C C' C'' D D' D'' : Container}
    (mu : ContainerMorphism C C') (mu' : ContainerMorphism C' C'')
    (nu : ContainerMorphism D D') (nu' : ContainerMorphism D' D'') :
    Container.coprod2 (mu.comp mu') (nu.comp nu')
      = (Container.coprod2 mu nu).comp (Container.coprod2 mu' nu')
```

Haskell.
```
-- coprod2 (mu `compM` mu') (nu `compM` nu')
--   = coprod2 mu nu `compM` coprod2 mu' nu'
```

3.4 Definition: `Container.coprodAssociator`

Definition 3.4 (Coproduct associator)**.** The isomorphism $(C + D) + E \cong C + (D + E)$:

- *Shapes*: Sum-reassociation $(\mathrm{inl}\,(\mathrm{inl}\,a) \mapsto \mathrm{inl}\,a$, etc.).
- *Positions*: identity in every branch.

Lean.

```
def Container.coprodAssociator (C D E : Container) :
    ContainerIso ((C.coprod D).coprod E) (C.coprod (D.coprod E)) where
  hom :=
    { onShapes := fun x ⇒ match x with
        | Sum.inl (Sum.inl a) ⇒ Sum.inl a
        | Sum.inl (Sum.inr b) ⇒ Sum.inr (Sum.inl b)
        | Sum.inr c           ⇒ Sum.inr (Sum.inr c)
      onPos := fun x ⇒ match x with
        | Sum.inl (Sum.inl _) ⇒ id
        | Sum.inl (Sum.inr _) ⇒ id
        | Sum.inr _           ⇒ id }
  inv :=
    { onShapes := fun x ⇒ match x with
        | Sum.inl a           ⇒ Sum.inl (Sum.inl a)
        | Sum.inr (Sum.inl b) ⇒ Sum.inl (Sum.inr b)
        | Sum.inr (Sum.inr c) ⇒ Sum.inr c
      onPos := fun x ⇒ match x with
        | Sum.inl _           ⇒ id
        | Sum.inr (Sum.inl _) ⇒ id
        | Sum.inr (Sum.inr _) ⇒ id }
  hom_inv := ...   -- proof omitted
  inv_hom := ...
```

Haskell.

```
coprodAssociator :: ContainerIso
  (Coprod (Coprod c d) e) (Coprod c (Coprod d e))
coprodAssociator = ...
-- Shapes: standard Either reassociation; positions: id everywhere.
```

3.5 Definition: `Container.coprodLeftUnitor`

Definition 3.5 (Coproduct left unitor)**.** The isomorphism $0 + C \cong C$: nothing in the left summand.

- Forward: $\mathrm{inl}\,e \mapsto e.\mathrm{elim}$ (impossible case); $\mathrm{inr}\,s \mapsto s$.
- Backward: $s \mapsto \mathrm{inr}\,s$.

Lean.

```
def Container.coprodLeftUnitor (C : Container) :
    ContainerIso (Container.zer ∘ coprod C) C where
  hom :=
    { onShapes := fun x ⇒ match x with | Sum.inl e ⇒ e.elim | Sum.
       inr s ⇒ s
      onPos    := fun x ⇒ match x with | Sum.inl e ⇒ e.elim | Sum.
         inr _ ⇒ id }
  inv :=
    { onShapes := Sum.inr
      onPos    := fun _ ⇒ id }
  hom_inv := ...;  inv_hom := ...
```

Haskell.
```
coprodLeftUnitor :: ContainerIso (Coprod Zero c) c
-- fwd: Left (absurd) or Right s → s
-- bwd: s → Right s
```

3.6 Definition: `Container.coprodRightUnitor`

Definition 3.6 (Coproduct right unitor)**.** The isomorphism $C + 0 \cong C$: nothing in the right summand.

Lean.
```
def Container.coprodRightUnitor (C : Container) :
    ContainerIso (C.coprod Container.zero) C where
  hom :=
    { onShapes := fun x ⇒ match x with | Sum.inl s ⇒ s | Sum.inr e
        ⇒ e.elim
      onPos    := fun x ⇒ match x with | Sum.inl _ ⇒ id | Sum.inr e
        ⇒ e.elim }
  inv := { onShapes := Sum.inl; onPos := fun _ ⇒ id }
  hom_inv := ...;  inv_hom := ...
```

Haskell.
```
coprodRightUnitor :: ContainerIso (Coprod c Zero) c
```

3.7 Theorem: `Container.coprodAssociator_naturality`

Theorem 3.7 (Naturality of coproduct associator)**.**

$$(\mathsf{coprod}_2\,(\mathsf{coprod}_2\,f_1\,f_2)\,f_3)\cdot\alpha^{+}_{Y_1,Y_2,Y_3} \;=\; \alpha^{+}_{X_1,X_2,X_3}\cdot\mathsf{coprod}_2\,f_1\,(\mathsf{coprod}_2\,f_2\,f_3).$$

Lean.
```
theorem Container.coprodAssociator_naturality
    {X1 Y1 X2 Y2 X3 Y3 : Container}
    (f1 : ContainerMorphism X1 Y1) (f2 : ContainerMorphism X2 Y2)
    (f3 : ContainerMorphism X3 Y3) :
    (Container.coprod2 (Container.coprod2 f1 f2) f3).comp
        (Container.coprodAssociator Y1 Y2 Y3).hom
      = (Container.coprodAssociator X1 X2 X3).hom.comp
          (Container.coprod2 f1 (Container.coprod2 f2 f3))
```

Haskell.
```
-- coprodAssociator_naturality: naturality square for coprodAssociator
```

3.8 Theorem: `Container.coprodLeftUnitor_naturality`

Theorem 3.8 (Naturality of coproduct left unitor)**.**

$$(\mathsf{coprod}_2\,(\mathrm{id}_0)\,f)\cdot\lambda^{+}_{Y} \;=\; \lambda^{+}_{X}\cdot f.$$

Lean.
```
theorem Container.coprodLeftUnitor_naturality {X Y : Container}
    (f : ContainerMorphism X Y) :
    (Container.coprod2 (ContainerMorphism.id Container.zero) f).comp
        (Container.coprodLeftUnitor Y).hom
      = (Container.coprodLeftUnitor X).hom.comp f
```

3.9 Theorem: `Container.coprodRightUnitor_naturality`

Theorem 3.9 (Naturality of coproduct right unitor)**.**

$$(\mathtt{coprod}_2\, f\, (\mathrm{id}_0)) \cdot \rho^+_Y \;=\; \rho^+_X \cdot f.$$

Lean.
```
theorem Container.coprodRightUnitor_naturality {X Y : Container}
    (f : ContainerMorphism X Y) :
    (Container.coprod2 f (ContainerMorphism.id Container.zero)).comp
        (Container.coprodRightUnitor Y).hom
      = (Container.coprodRightUnitor X).hom.comp f
```

3.10 Theorem: `Container.coprodPentagon`

Theorem 3.10 (Pentagon for $+$)**.** The two re-association paths of $((C_1 + C_2) + C_3) + C_4 \to C_1 + (C_2 + (C_3 + C_4))$ agree.

Lean.
```
theorem Container.coprodPentagon (C1 C2 C3 C4 : Container) :
    ((Container.coprod2 (Container.coprodAssociator C1 C2 C3).hom
          (ContainerMorphism.id C4)).comp
        (Container.coprodAssociator C1 (C2.coprod C3) C4).hom).comp
        (Container.coprod2 (ContainerMorphism.id C1)
          (Container.coprodAssociator C2 C3 C4).hom)
      = (Container.coprodAssociator (C1.coprod C2) C3 C4).hom.comp
          (Container.coprodAssociator C1 C2 (C3.coprod C4)).hom
```

3.11 Theorem: `Container.coprodTriangle`

Theorem 3.11 (Triangle for $+$)**.**

$$\alpha^+_{C_1,0,C_2} \cdot (\mathrm{id}_{C_1} + \lambda^+_{C_2}) \;=\; \rho^+_{C_1} + \mathrm{id}_{C_2}.$$

Lean.
```
theorem Container.coprodTriangle (C1 C2 : Container) :
    (Container.coprodAssociator C1 Container.zero C2).hom.comp
        (Container.coprod2 (ContainerMorphism.id C1)
          (Container.coprodLeftUnitor C2).hom)
      = Container.coprod2 (Container.coprodRightUnitor C1).hom
          (ContainerMorphism.id C2)
```

3.12 Definition: `contCoprodMonoidal`

Definition 3.12 ($(\mathbf{Cont}, +, 0)$ is a monoidal category)**.** A named `def` (not an `instance`): the coproduct $+$ with morphism action $\mathtt{coprod}_2$, unit 0, and the structural isomorphisms above satisfy all monoidal-category axioms.

Lean.
```
@[reducible] def contCoprodMonoidal : MonoidalCategory Container where
  tensorObj          := Container.coprod
  tensorHom          := Container.coprod2
  tensorHom_id       := Container.coprod2_id
  tensorHom_comp     := fun f1 g1 f2 g2 ⇒ Container.coprod2_comp f1 g1
      f2 g2
  tensorUnit         := Container.zero
  associator         := fun X Y Z ⇒ (Container.coprodAssociator X Y Z)
     .toCatIso
```

```
  leftUnitor         := fun X ⇒ (Container.coprodLeftUnitor X).
     toCatIso
  rightUnitor        := fun X ⇒ (Container.coprodRightUnitor X).
     toCatIso
  associator_naturality  :=
    fun f1 f2 f3 ⇒ Container.coprodAssociator_naturality f1 f2 f3
  leftUnitor_naturality  := fun f ⇒ Container.
     coprodLeftUnitor_naturality f
  rightUnitor_naturality := fun f ⇒ Container.
     coprodRightUnitor_naturality f
  pentagon           := Container.coprodPentagon
  triangle           := Container.coprodTriangle
```

Haskell.

```
contCoprodMonoidal :: MonoidalCategoryDef ContainerMorph
contCoprodMonoidal = MonoidalCategoryDef
  { tensor = coprod2, unit = zeroContainer, ... }
```

4 The categorical product $(\times, 1)$

4.1 Definition: `Container.prod₂`

Definition 4.1 (Bifunctorial action of $\times$)**.** The morphism action $\texttt{prod}_2\,\mu\,\nu : C \times D \Rightarrow C' \times D'$ is derived from the product's universal property: pair the compositions $\pi_1 \cdot \mu$ and $\pi_2 \cdot \nu$.

Lean.

```
def Container.prod2 {C C' D D' : Container}
    (mu : ContainerMorphism C C') (nu : ContainerMorphism D D') :
    ContainerMorphism (C.prod D) (C'.prod D') :=
  ((Container.fst C D).comp mu).pair ((Container.snd C D).comp nu)
```

Haskell.

```
prod2 :: Morph c p c' p' → Morph d q d' q'
      → Morph (c `Prod` d) (Either p q) (c' `Prod` d') (Either p' q')
prod2 mu nu = pairMorph (fstMorph `compM` mu) (sndMorph `compM` nu)
```

4.2 Theorem: `Container.prod₂_id`

Theorem 4.2 (Product bifunctoriality: identity)**.** $\texttt{prod}_2\,(\mathrm{id}_C)\,(\mathrm{id}_D) = \mathrm{id}_{C \times D}$.

Lean.

```
theorem Container.prod2_id (C D : Container) :
    Container.prod2 (ContainerMorphism.id C) (ContainerMorphism.id D)
      = ContainerMorphism.id (C.prod D)
```

4.3 Theorem: `Container.prod₂_comp`

Theorem 4.3 (Product bifunctoriality: interchange)**.** $\texttt{prod}_2\,(\mu \cdot \mu')\,(\nu \cdot \nu') = \texttt{prod}_2\,\mu\,\nu \cdot \texttt{prod}_2\,\mu'\,\nu'$.

Lean.

```
theorem Container.prod2_comp {C C' C'' D D' D'' : Container}
    (mu : ContainerMorphism C C') (mu' : ContainerMorphism C' C'')
    (nu : ContainerMorphism D D') (nu' : ContainerMorphism D' D'') :
    Container.prod2 (mu.comp mu') (nu.comp nu')
      = (Container.prod2 mu nu).comp (Container.prod2 mu' nu')
```

4.4 Definition: `Container.prodAssociator`

Definition 4.4 (Product associator)**.** The isomorphism $(C \times D) \times E \cong C \times (D \times E)$:

- *Shapes*: Prod-reassociation $((s,t),u) \mapsto (s,(t,u))$.
- *Positions*: Sum-reassociation *backwards* (since position maps are contravariant); the backward direction reassociates $P\,s \sqcup (Q\,t \sqcup R\,u)$ to $(P\,s \sqcup Q\,t) \sqcup R\,u$.

Lean.

```
def Container.prodAssociator (C D E : Container) :
    ContainerIso ((C.prod D).prod E) (C.prod (D.prod E)) where
  hom :=
    { onShapes := fun x ⇒ (x.1.1, (x.1.2, x.2))
      onPos    := fun _ p ⇒
        Sum.elim (fun a ⇒ Sum.inl (Sum.inl a))
          (Sum.elim (fun b ⇒ Sum.inl (Sum.inr b))
                    (fun c ⇒ Sum.inr c)) p }
  inv :=
    { onShapes := fun x ⇒ ((x.1, x.2.1), x.2.2)
      onPos    := fun _ p ⇒
        Sum.elim (Sum.elim (fun a ⇒ Sum.inl a)
                           (fun b ⇒ Sum.inr (Sum.inl b)))
                 (fun c ⇒ Sum.inr (Sum.inr c)) p }
  hom_inv := ...;  inv_hom := ...
```

Haskell.

```
-- Shapes: reassociate tuples.
-- Positions: reassociate Either (backwards, since contravariant).
prodAssociator :: ContainerIso (Prod (Prod c d) e) (Prod c (Prod d e))
```

Fidelity note. The positions of $C \times D$ are $P\,s \sqcup Q\,t$. The associator's position map goes *backward*: given a position of $C \times (D \times E)$, it produces a position of $(C \times D) \times E$. The backward position map of the forward direction is a *rightward* Sum-reassociation (injecting left-nested sums into right-nested ones), which is what the code shows.

4.5 Definition: `Container.prodLeftUnitor`

Definition 4.5 (Product left unitor)**.** The isomorphism $1 \times C \cong C$. Since 1 has shape $\mathbf{1}$ and no positions, the position type of $1 \times C$ at shape $(*, s)$ is $\emptyset \sqcup P\,s \cong P\,s$.

- Forward on shapes: $(*, s) \mapsto s$.
- Forward on positions: inject right, $p \mapsto \mathrm{inr}\,p$.

Lean.

```
def Container.prodLeftUnitor (C : Container) :
    ContainerIso (Container.one.prod C) C where
  hom :=
    { onShapes := fun x ⇒ x.2
      onPos    := fun _ p ⇒ Sum.inr p }
  inv :=
    { onShapes := fun s ⇒ ((), s)
      onPos    := fun _ p ⇒ Sum.elim (fun e ⇒ e.elim) id p }
  hom_inv := ...;  inv_hom := ...
```

4.6 Definition: `Container.prodRightUnitor`

Definition 4.6 (Product right unitor)**.** The isomorphism $C \times 1 \cong C$:

- Forward on shapes: $(s, *) \mapsto s$.
- Forward on positions: inject left, $p \mapsto \mathrm{inl}\, p$.

Lean.
```
def Container.prodRightUnitor (C : Container) :
    ContainerIso (C.prod Container.one) C where
  hom :=
    { onShapes := fun x ⇒ x.1
      onPos    := fun _ p ⇒ Sum.inl p }
  inv :=
    { onShapes := fun s ⇒ (s, ())
      onPos    := fun _ p ⇒ Sum.elim id (fun e ⇒ e.elim) p }
  hom_inv := ...;  inv_hom := ...
```

4.7 Theorem: `Container.prodAssociator_naturality`

Theorem 4.7 (Naturality of product associator)**.**

$$(\mathtt{prod}_2\,(\mathtt{prod}_2\, f_1\, f_2)\, f_3) \cdot \alpha^{\times}_{Y_1,Y_2,Y_3} \;=\; \alpha^{\times}_{X_1,X_2,X_3} \cdot \mathtt{prod}_2\, f_1\, (\mathtt{prod}_2\, f_2\, f_3).$$

Lean.
```
theorem Container.prodAssociator_naturality
    {X1 Y1 X2 Y2 X3 Y3 : Container}
    (f1 : ContainerMorphism X1 Y1) (f2 : ContainerMorphism X2 Y2)
    (f3 : ContainerMorphism X3 Y3) :
    (Container.prod2 (Container.prod2 f1 f2) f3).comp
        (Container.prodAssociator Y1 Y2 Y3).hom
      = (Container.prodAssociator X1 X2 X3).hom.comp
          (Container.prod2 f1 (Container.prod2 f2 f3))
```

4.8 Theorem: `Container.prodLeftUnitor_naturality`

Theorem 4.8 (Naturality of product left unitor)**.** $(\mathtt{prod}_2\,(\mathrm{id}_1)\, f) \cdot \lambda^{\times}_Y = \lambda^{\times}_X \cdot f$.

Lean.
```
theorem Container.prodLeftUnitor_naturality {X Y : Container}
    (f : ContainerMorphism X Y) :
    (Container.prod2 (ContainerMorphism.id Container.one) f).comp
        (Container.prodLeftUnitor Y).hom
      = (Container.prodLeftUnitor X).hom.comp f := rfl
```

4.9 Theorem: `Container.prodRightUnitor_naturality`

Theorem 4.9 (Naturality of product right unitor)**.** $(\mathtt{prod}_2\, f\,(\mathrm{id}_1)) \cdot \rho^{\times}_Y = \rho^{\times}_X \cdot f$.

Lean.
```
theorem Container.prodRightUnitor_naturality {X Y : Container}
    (f : ContainerMorphism X Y) :
    (Container.prod2 f (ContainerMorphism.id Container.one)).comp
        (Container.prodRightUnitor Y).hom
      = (Container.prodRightUnitor X).hom.comp f := rfl
```

4.10 Theorem: `Container.prodPentagon`

Theorem 4.10 (Pentagon for $\times$)**.** The two re-association paths of $((C_1 \times C_2) \times C_3) \times C_4 \to C_1 \times (C_2 \times (C_3 \times C_4))$ agree.

Lean.

```
theorem Container.prodPentagon (C1 C2 C3 C4 : Container) :
    ((Container.prod2 (Container.prodAssociator C1 C2 C3).hom
          (ContainerMorphism.id C4)).comp
        (Container.prodAssociator C1 (C2.prod C3) C4).hom).comp
        (Container.prod2 (ContainerMorphism.id C1)
          (Container.prodAssociator C2 C3 C4).hom)
      = (Container.prodAssociator (C1.prod C2) C3 C4).hom.comp
          (Container.prodAssociator C1 C2 (C3.prod C4)).hom
```

4.11 Theorem: `Container.prodTriangle`

Theorem 4.11 (Triangle for $\times$)**.** $\alpha^{\times}_{C_1,1,C_2} \cdot (\mathrm{id}_{C_1} \times \lambda^{\times}_{C_2}) = \rho^{\times}_{C_1} \times \mathrm{id}_{C_2}$.

Lean.

```
theorem Container.prodTriangle (C1 C2 : Container) :
    (Container.prodAssociator C1 Container.one C2).hom.comp
        (Container.prod2 (ContainerMorphism.id C1)
          (Container.prodLeftUnitor C2).hom)
      = Container.prod2 (Container.prodRightUnitor C1).hom
          (ContainerMorphism.id C2)
```

4.12 Definition: `contProdMonoidal`

Definition 4.12 ((**Cont**, $\times$, 1) is a monoidal category)**.** A named `def`: the categorical product $\times$ with morphism action $\texttt{prod}_2$, unit 1, and the structural isomorphisms satisfy all monoidal-category axioms.

Lean.

```
@[reducible] def contProdMonoidal : MonoidalCategory Container where
  tensorObj          := Container.prod
  tensorHom          := Container.prod2
  tensorHom_id       := Container.prod2_id
  tensorHom_comp     := fun f1 g1 f2 g2 ⇒ Container.prod2_comp f1 g1
     f2 g2
  tensorUnit         := Container.one
  associator         := fun X Y Z ⇒ (Container.prodAssociator X Y Z).
     toCatIso
  leftUnitor         := fun X ⇒ (Container.prodLeftUnitor X).toCatIso
  rightUnitor        := fun X ⇒ (Container.prodRightUnitor X).toCatIso
  associator_naturality  :=
    fun f1 f2 f3 ⇒ Container.prodAssociator_naturality f1 f2 f3
  leftUnitor_naturality  := fun f ⇒ Container.
     prodLeftUnitor_naturality f
  rightUnitor_naturality := fun f ⇒ Container.
     prodRightUnitor_naturality f
  pentagon           := Container.prodPentagon
  triangle           := Container.prodTriangle
```

5 The extension functor as a strong monoidal functor

5.1 Definitions and theorems: coproduct comparison

Definition 5.1 (Coproduct comparison isomorphism)**.** The maps

$$\texttt{coprodExt} : [\![C + D]\!]X \xrightarrow{\sim} [\![C]\!]X \sqcup [\![D]\!]X,$$

$$\texttt{coprodExtInv} : [\![C]\!]X \sqcup [\![D]\!]X \xrightarrow{\sim} [\![C + D]\!]X.$$

A $(C + D)$-structure is a C-structure or a D-structure, according to the injection its shape came from. Both round trips hold.

Lean.

```
def Container.coprodExt {C D : Container} {X : Type} :
    Ext (C.coprod D) X → Ext C X |+| Ext D X :=
  fun p ⇒ match p with
    | ⟨Sum.inl s, f⟩ ⇒ Sum.inl ⟨s, f⟩
    | ⟨Sum.inr t, f⟩ ⇒ Sum.inr ⟨t, f⟩

def Container.coprodExtInv {C D : Container} {X : Type} :
    Ext C X |+| Ext D X → Ext (C.coprod D) X :=
  fun p ⇒ match p with
    | Sum.inl q ⇒ ⟨Sum.inl q.1, q.2⟩
    | Sum.inr q ⇒ ⟨Sum.inr q.1, q.2⟩

theorem Container.coprodExtInv_coprodExt {C D : Container} {X : Type}
    (p : Ext (C.coprod D) X) :
    Container.coprodExtInv (Container.coprodExt p) = p

theorem Container.coprodExt_coprodExtInv {C D : Container} {X : Type}
    (p : Ext C X |+| Ext D X) :
    Container.coprodExt (Container.coprodExtInv p) = p
```

Haskell.

```
coprodExt :: Ext (Coprod c d) x → Either (Ext c x) (Ext d x)
coprodExt (Ext (Left s)  f) = Left  (Ext s f)
coprodExt (Ext (Right t) f) = Right (Ext t f)

coprodExtInv :: Either (Ext c x) (Ext d x) → Ext (Coprod c d) x
coprodExtInv (Left  (Ext s f)) = Ext (Left  s) f
coprodExtInv (Right (Ext t f)) = Ext (Right t) f
```

5.2 Definition/Theorem: `Container.zeroExt`

Definition 5.2 (Unit comparison for 0)**.** The map $[\![0]\!]X \to \emptyset$: there is no shape, so there is no structure. The round trip is trivial (`Empty.elim`).

Lean.

```
def Container.zeroExt {X : Type} : Ext Container.zero X → Empty :=
  fun p ⇒ p.1.elim

theorem Container.zeroExt_iso {X : Type} (p : Ext Container.zero X) :
    (Container.zeroExt p).elim = p := p.1.elim
```

Haskell.

```
zeroExt :: Ext ZeroContainer x → Void
zeroExt (Ext s _) = absurd s
```

5.3 Definitions and theorems: product comparison

Definition 5.3 (Product comparison isomorphism)**.** The maps

$$\texttt{prodExt} : [\![C \times D]\!]X \xrightarrow{\sim} [\![C]\!]X \times [\![D]\!]X,$$

$$\texttt{prodExtInv} : [\![C]\!]X \times [\![D]\!]X \xrightarrow{\sim} [\![C \times D]\!]X.$$

Since positions of $C \times D$ are $P\,s \sqcup Q\,t$, a labelling $P\,s \sqcup Q\,t \to X$ is a pair of labellings $(P\,s \to X, Q\,t \to X)$ by the exponential law $y^{A+B} \cong y^A \times y^B$. This is why the product container has the *pointwise product* of extensions.

Lean.

```
def Container.prodExt {C D : Container} {X : Type} :
    Ext (C.prod D) X → Ext C X * Ext D X :=
  fun p ⇒ (⟨p.1.1, fun i ⇒ p.2 (Sum.inl i)⟩,
           ⟨p.1.2, fun j ⇒ p.2 (Sum.inr j)⟩)

def Container.prodExtInv {C D : Container} {X : Type} :
    Ext C X * Ext D X → Ext (C.prod D) X :=
  fun p ⇒ ⟨(p.1.1, p.2.1), Sum.elim p.1.2 p.2.2⟩

theorem Container.prodExtInv_prodExt {C D : Container} {X : Type}
    (p : Ext (C.prod D) X) :
    Container.prodExtInv (Container.prodExt p) = p

theorem Container.prodExt_prodExtInv {C D : Container} {X : Type}
    (p : Ext C X * Ext D X) :
    Container.prodExt (Container.prodExtInv p) = p
```

Haskell.

```
prodExt :: Ext (Prod c d) x → (Ext c x, Ext d x)
prodExt (Ext (s, t) f) =
  (Ext s (\i → f (Left i)), Ext t (\j → f (Right j)))

prodExtInv :: (Ext c x, Ext d x) → Ext (Prod c d) x
prodExtInv (Ext s f, Ext t g) = Ext (s, t) (either f g)
```

5.4 Definitions and theorems: unit comparison for 1

Definition 5.4 (Unit comparison for 1)**.** The isomorphism $[\![1]\!]X \cong \mathbf{1}$: one shape, no positions, so exactly one structure.

Lean.

```
def Container.oneExt {X : Type} : Ext Container.one X → Unit :=
  fun _ ⇒ ()

def Container.oneExtInv {X : Type} : Unit → Ext Container.one X :=
  fun _ ⇒ ⟨(), fun e ⇒ e.elim⟩

theorem Container.oneExtInv_oneExt {X : Type}
    (p : Ext Container.one X) :
    Container.oneExtInv (Container.oneExt p) = p

theorem Container.oneExt_oneExtInv {X : Type} (u : Unit) :
    Container.oneExt (Container.oneExtInv (X := X) u) = u := rfl
```

Haskell.

```
oneExt    :: Ext OneContainer x → ()
oneExt _  = ()

oneExtInv :: () → Ext OneContainer x
oneExtInv () = Ext () absurd
```

5.5 Definitions: shared unit comparison $[\![y]\!]X \cong X$

Definition 5.5 (Unit comparison for $y = I$)**.** The isomorphism $[\![y]\!]X \cong X$: read the label at the unique position. This is the unit comparison for both $\otimes$ and $\triangleleft$, since $y = I$.

$$\texttt{yExt}(s, v) = v(*), \qquad \texttt{yExtInv}(x) = (*,\ _ \mapsto x).$$

Lean.

```
def Container.yExt {X : Type} : Ext Container.y X → X :=
  fun p ⇒ p.2 ()

def Container.yExtInv {X : Type} : X → Ext Container.y X :=
  fun x ⇒ ⟨(), fun _ ⇒ x⟩

theorem Container.yExtInv_yExt {X : Type}
    (p : Ext Container.y X) :
    Container.yExtInv (Container.yExt p) = p := rfl

theorem Container.yExt_yExtInv {X : Type} (x : X) :
    Container.yExt (Container.yExtInv x) = x := rfl
```

Haskell.

```
yExt :: Ext YContainer x → x
yExt (Ext () v) = v ()

yExtInv :: x → Ext YContainer x
yExtInv x = Ext () (const x)
```

Fidelity note. Both round trips hold by `rfl`: definitional η for `Unit` and the `Sigma` / `Ext` type.